数字数据结构竞赛板子

本板子默认 C++17、GCC/Clang，统一使用 `ll`、`i128`。目标是处理普通整数容易溢出的精确计算：大整数、精确分数、模整数和高精度浮点。

0. 公共约定与选择表

```cpp
#include <bits/stdc++.h>
#include <boost/multiprecision/cpp_int.hpp>
#include <boost/multiprecision/cpp_dec_float.hpp>
using namespace std;
using ll = long long;
using i128 = __int128_t;
using u128 = __uint128_t;
using boost::multiprecision::cpp_int;
```

需求	推荐类型	说明
`ll` 乘加可能溢出	`i128`	最快，范围约 `+-10^38`
任意长度精确整数	`cpp_int`	首选，功能全且大数算法经过优化
禁止 Boost、数位不太多	自写 `Big`	基数 `10^9`，接口完整、容易裁剪
固定质数模四则运算	`Mint<MOD>`	编译期模数，常数小
运行时模数、模数可合数	`DMod`	`i128` 模乘，逆元用扩展 gcd
分子分母能放进 `ll`	`Frac`	交叉约分，比较用 `i128`
任意长度精确分数	`BigFrac`	分子分母使用 `cpp_int`
约 50 至数百位浮点	`cpp_dec_float`	近似值，不是精确十进制有理数

能用 `i128` 时不要上大整数；能用 `cpp_int` 时通常不要现场手写完整高精度。自写 `Big` 主要用于环境禁用 Boost 或只想复制必要接口。

---

1. `i128` 输入输出与安全整数运算

`i128` 没有标准流运算符。下方支持带符号十进制输入输出、有符号向下/向上整除，以及 `ll` 范围模乘。

- `read128()`：从 `cin` 读取一个 `i128`。
- `str128(x)`、`print128(x)`：转字符串、直接输出。
- `floor_div(a,b)`、`ceil_div(a,b)`：数学意义的向下/向上取整，要求 `b!=0`。
- `mul_mod(a,b,mod)`：`0<=a,b<mod<=LLONG_MAX` 时安全计算乘积取模。

```cpp
i128 read128() {
    string s; cin >> s;
    int p = 0, sg = 1;
    if (s[p] == '+' || s[p] == '-') sg = s[p++] == '-' ? -1 : 1;
    i128 x = 0;
    for (; p < (int)s.size(); p++) x = x * 10 + s[p] - '0';
    return sg == 1 ? x : -x;
}

string str128(i128 x) {
    if (!x) return "0";
    bool neg = x < 0;
    u128 v = neg ? 0 - (u128)x : (u128)x;
    string s;
    while (v) s.push_back('0' + v % 10), v /= 10;
    if (neg) s.push_back('-');
    reverse(s.begin(), s.end());
    return s;
}

void print128(i128 x) { cout << str128(x); }

i128 floor_div(i128 a, i128 b) { // floor(a/b), b!=0
    i128 q = a / b, r = a % b;
    if (r && ((r > 0) != (b > 0))) q--;
    return q;
}

i128 ceil_div(i128 a, i128 b) { // ceil(a/b), b!=0
    i128 q = a / b, r = a % b;
    if (r && ((r > 0) == (b > 0))) q++;
    return q;
}

ll mul_mod(ll a, ll b, ll mod) { // 0<=a,b<mod
    return (i128)a * b % mod;
}
```

`str128` 用无符号绝对值处理最小负数，避免直接执行 `-x` 溢出。`i128` 仍是定长类型，连续乘法前要估计最终范围。

---

2. Boost 高精度整数 `cpp_int`

2.1 基本用法

`cpp_int` 支持任意长度带符号整数的输入输出、比较、四则运算、取模和位运算。乘法会按规模自动选择更快算法，通常比临时写的朴素高精度更可靠。

```cpp
using BigInt = cpp_int;
```

```
BigInt a, b;
cin >> a >> b;
cout << a + b << '\n';
cout << a - b << '\n';
cout << a * b << '\n';
cout << a / b << ' ' << a % b << '\n'; // b!=0，商向 0 截断

BigInt x = 1;
x <<= 1000;                                  // 2^1000
bool bit = boost::multiprecision::bit_test(x, 1000);
int hi = boost::multiprecision::msb(x);  // 最高位编号，要求 x>0
string s = x.convert_to<string>();
```

不要把超出范围的 `cpp_int` 直接 `convert_to<ll>()`；该转换不会替你完成可靠的范围检查。需要时先与 `numeric_limits<ll>::min/max()` 比较。

2.2 gcd、幂与整数平方根

- `gcd_big(a,b)`：非负 gcd。
- `pow_big(a,e)`：精确计算 `a^e`，指数是 `uint64_t`。
- `boost::multiprecision::powm(a,e,mod)`：高精度快速幂取模。
- `isqrt(n)`：返回 `floor(sqrt(n))`，要求 `n>=0`。

```
BigInt abs_big(BigInt x) { return x < 0 ? -x : x; }

BigInt gcd_big(BigInt a, BigInt b) {
    a = abs_big(a), b = abs_big(b);
    while (b != 0) a %= b, swap(a, b);
    return a;
}

BigInt pow_big(BigInt a, uint64_t e) {
    BigInt r = 1;
    for (; e; e >>= 1, a *= a) if (e & 1) r *= a;
    return r;
}

BigInt isqrt(const BigInt &n) { // floor(sqrt(n)), n>=0
    assert(n >= 0);
    if (n == 0) return 0;
    BigInt x = BigInt(1) << ((boost::multiprecision::msb(n) + 2) / 2);
    while (true) {
        BigInt y = (x + n / x) >> 1;
        if (y >= x) return x;
        x = y;
    }
}
```

精确组合数、阶乘、斐波那契等只需把普通整数类型替换成 `BigInt`。但循环次数和结果位数都很大时，算法复杂度仍然是主要瓶颈。

---

3. 自写大整数 `Big`：禁止 Boost 时使用

带符号、基数 `10^9`，支持输入输出、比较、正负号、加减乘除、取模以及与小整数运算。每个 `int` 保存 9 位十进制，内存和常数远优于逐位存储。

- `Big x(s)`：从十进制字符串构造。
- 四则运算与 `%` 的语义和 C++ 有符号整数一致：商向 `0` 截断，余数与被除数同号。
- 大整数乘法为 `O(nm)`，除法为归一化长除法 `O(nm)`；数千位以上优先使用 `cpp_int`。
- 除数不能为 `0`。内部会自动去前导零，零的符号始终为正。

```
struct Big {
    static const int BASE = 1000000000;
    static const int DIG = 9;
    vector<int> a; // little-endian base 1e9
    int sg = 1;

    Big(ll v = 0) { *this = v; }
    Big(const string &s) { read(s); }

    Big &operator=(ll v) {
        a.clear(), sg = 1;
        uint64_t x = v < 0 ? 0 - (uint64_t)v : (uint64_t)v;
        if (v < 0) sg = -1;
        while (x) a.push_back(x % BASE), x /= BASE;
        return *this;
    }
    bool zero() const { return a.empty(); }
    void trim() {
        while (!a.empty() && !a.back()) a.pop_back();
        if (a.empty()) sg = 1;
    }
    static int cmp_abs(const Big &x, const Big &y) {
        if (x.a.size() != y.a.size()) return x.a.size() < y.a.size() ? -1 : 1;
        for (int i = (int)x.a.size() - 1; i >= 0; i--)
            if (x.a[i] != y.a[i]) return x.a[i] < y.a[i] ? -1 : 1;
        return 0;
    }
    Big abs() const { Big x = *this; x.sg = 1; return x; }
    Big operator-() const {
        Big x = *this;
        if (!x.zero()) x.sg = -x.sg;
        return x;
    }
    void add_abs(const Big &o) {
        int n = max(a.size(), o.a.size());
        a.resize(n);
```

```
        ll c = 0;
        for (int i = 0; i < n || c; i++) {
            if (i == (int)a.size()) a.push_back(0);
            ll x = (ll)a[i] + c + (i < (int)o.a.size() ? o.a[i] : 0);
            a[i] = x % BASE, c = x / BASE;
        }
    }
    void sub_abs(const Big &o) { // |*this|>=|o|
        int c = 0;
        for (int i = 0; i < (int)a.size(); i++) {
            ll x = (ll)a[i] - c - (i < (int)o.a.size() ? o.a[i] : 0);
            c = x < 0;
            if (c) x += BASE;
            a[i] = x;
        }
        trim();
    }
    Big &operator+=(const Big &o) {
        if (sg == o.sg) add_abs(o);
        else {
            int c = cmp_abs(*this, o);
            if (c >= 0) sub_abs(o);
            else { Big x = o; x.sub_abs(*this); *this = move(x); }
        }
        return *this;
    }
    Big &operator-=(const Big &o) { return *this += -o; }
    friend Big operator+(Big x, const Big &y) { return x += y; }
    friend Big operator-(Big x, const Big &y) { return x -= y; }

    Big &operator*=(const Big &o) {
        if (zero() || o.zero()) return *this = 0;
        vector<int> c(a.size() + o.a.size() + 1);
        for (int i = 0; i < (int)a.size(); i++) {
            ll carry = 0;
            for (int j = 0; j < (int)o.a.size() || carry; j++) {
                i128 x = c[i + j] + carry;
                if (j < (int)o.a.size()) x += (i128)a[i] * o.a[j];
                c[i + j] = x % BASE, carry = x / BASE;
            }
        }
        a = move(c), sg *= o.sg, trim();
        return *this;
    }
    Big &operator*=(int v) {
        if (!v || zero()) return *this = 0;
        ll x = v;
        if (x < 0) sg = -sg, x = -x;
        ll carry = 0;
        for (int i = 0; i < (int)a.size() || carry; i++) {
            if (i == (int)a.size()) a.push_back(0);
            i128 z = (i128)a[i] * x + carry;
```

```
            a[i] = z % BASE, carry = z / BASE;
        }
        trim();
        return *this;
    }
    friend Big operator*(Big x, const Big &y) { return x *= y; }
    friend Big operator*(Big x, int y) { return x *= y; }
    friend Big operator*(int y, Big x) { return x *= y; }

    Big &operator/=(int v) {
        assert(v != 0);
        ll x = v;
        if (x < 0) sg = -sg, x = -x;
        ll rem = 0;
        for (int i = (int)a.size() - 1; i >= 0; i--) {
            i128 cur = (i128)rem * BASE + a[i];
            a[i] = cur / x, rem = cur % x;
        }
        trim();
        return *this;
    }
    ll mod_int(int v) const {
        assert(v != 0);
        ll x = v < 0 ? -(ll)v : (ll)v, rem = 0;
        for (int i = (int)a.size() - 1; i >= 0; i--)
            rem = ((i128)rem * BASE + a[i]) % x;
        return sg * rem;
    }
    friend Big operator/(Big x, int v) { return x /= v; }

    void shift(int x) { // *this = *this*BASE+x, 当前数非负
        if (a.empty() && !x) return;
        a.insert(a.begin(), x), trim();
    }
    friend pair<Big, Big> divmod(const Big &x, const Big &y) {
        assert(!y.zero());
        int norm = BASE / (y.a.back() + 1);
        Big a = x.abs() * norm, b = y.abs() * norm;
        Big q, r;
        q.a.assign(a.a.size(), 0);
        for (int i = (int)a.a.size() - 1; i >= 0; i--) {
            r.shift(a.a[i]);
            int m = b.a.size();
            ll s1 = (int)r.a.size() > m ? r.a[m] : 0;
            ll s2 = (int)r.a.size() >= m ? r.a[m - 1] : 0;
            int d = min((ll)BASE - 1, ((ll)BASE * s1 + s2) / b.a.back());
            r -= b * d;
            while (r.sg < 0) r += b, d--;
            q.a[i] = d;
        }
        q.sg = x.sg * y.sg, q.trim();
        r /= norm, r.sg = x.sg, r.trim();
```

```
        return {q, r};
    }
    Big &operator/=(const Big &o) { return *this = divmod(*this, o).first; }
    Big &operator%=(const Big &o) { return *this = divmod(*this, o).second; }
    friend Big operator/(Big x, const Big &y) { return x /= y; }
    friend Big operator%(Big x, const Big &y) { return x %= y; }

    friend bool operator==(const Big &x, const Big &y) { return x.sg == y.sg && x.a
== y.a; }
    friend bool operator!=(const Big &x, const Big &y) { return !(x == y); }
    friend bool operator<(const Big &x, const Big &y) {
        if (x.sg != y.sg) return x.sg < y.sg;
        int c = cmp_abs(x, y);
        return x.sg == 1 ? c < 0 : c > 0;
    }
    friend bool operator>(const Big &x, const Big &y) { return y < x; }
    friend bool operator<=(const Big &x, const Big &y) { return !(y < x); }
    friend bool operator>=(const Big &x, const Big &y) { return !(x < y); }

    void read(string s) {
        a.clear(), sg = 1;
        int p = 0;
        if (s[p] == '+' || s[p] == '-') sg = s[p++] == '-' ? -1 : 1;
        for (int i = (int)s.size(); i > p; i -= DIG) {
            int l = max(p, i - DIG), x = 0;
            for (int j = l; j < i; j++) x = x * 10 + s[j] - '0';
            a.push_back(x);
        }
        trim();
    }
    friend istream &operator>>(istream &in, Big &x) { string s; in >> s; x.read(s);
return in; }
    friend ostream &operator<<(ostream &out, const Big &x) {
        if (x.sg < 0) out << '-';
        if (x.a.empty()) return out << '0';
        out << x.a.back();
        char old = out.fill('0');
        for (int i = (int)x.a.size() - 2; i >= 0; i--) out << setw(DIG) << x.a[i];
        out.fill(old);
        return out;
    }
};

Big gcd(Big a, Big b) {
    a.sg = b.sg = 1;
    while (!b.zero()) a %= b, swap(a, b);
    return a;
}

Big pow_big(Big a, uint64_t e) {
    Big r = 1;
    for (; e; e >>= 1, a *= a) if (e & 1) r *= a;
```

```
    return r;
}
```

若只需要非负数，可以删除 `sg` 与所有符号分支；只需要加减乘小整数时，可删除约一半代码。不要把自写 `Big` 与 Boost 版的 `pow_big/gcd` 同时复制到同一文件，它们是重载友好但可能让 `0` 等字面量产生不必要的推导歧义，按题目保留所需版本即可。

---

4. 模整数

4.1 编译期质数模 `Mint`

适合 `998244353`、`1000000007` 等固定质数模。模数是模板参数，编译器能充分内联；乘法使用 `ll`，要求 `MOD<2^31`。

- `Mint<MOD> x(v)`：自动把有符号 `ll` 归一化到 `[0,MOD)`。
- 支持 `+ - * /`、复合赋值、比较和流输入输出。
- `pow(e)`：快速幂，`e>=0`。
- `inv()`、除法：使用费马小定理，要求 `MOD` 是质数且当前值非零。

```
template<int MOD> struct Mint {
    int x;
    Mint(ll v = 0) {
        v %= MOD;
        if (v < 0) v += MOD;
        x = v;
    }
    static constexpr int mod() { return MOD; }
    int val() const { return x; }
    Mint pow(ll e) const { // e>=0
        Mint a = *this, r = 1;
        for (; e; e >>= 1, a *= a) if (e & 1) r *= a;
        return r;
    }
    Mint inv() const { // MOD 为质数，x!=0
        assert(x != 0);
        return pow(MOD - 2);
    }
    Mint operator-() const { return Mint(x ? MOD - x : 0); }
    Mint &operator+=(const Mint &o) {
        ll y = (ll)x + o.x;
        x = y >= MOD ? y - MOD : y;
        return *this;
    }
    Mint &operator-=(const Mint &o) {
        x -= o.x;
        if (x < 0) x += MOD;
        return *this;
    }
```

```
    Mint &operator*=(const Mint &o) { x = (ll)x * o.x % MOD; return *this; }
    Mint &operator/=(const Mint &o) { return *this *= o.inv(); }
    friend Mint operator+(Mint a, const Mint &b) { return a += b; }
    friend Mint operator-(Mint a, const Mint &b) { return a -= b; }
    friend Mint operator*(Mint a, const Mint &b) { return a *= b; }
    friend Mint operator/(Mint a, const Mint &b) { return a /= b; }
    friend bool operator==(const Mint &a, const Mint &b) { return a.x == b.x; }
    friend bool operator!=(const Mint &a, const Mint &b) { return a.x != b.x; }
    friend istream &operator>>(istream &in, Mint &a) { ll v; in >> v; a = Mint(v);
return in; }
    friend ostream &operator<<(ostream &out, const Mint &a) { return out << a.x; }
};

using Z = Mint<998244353>;
```

合数模下不能用 `pow(MOD-2)` 求逆，应改用下一节的动态模数类，或明确判断 `gcd(x,MOD)=1` 后用扩展 gcd。

4.2 运行时任意 `ll` 模数 `DMod`

支持最高到 `LLONG_MAX` 的正模数，乘法用 `i128`。模数可以是合数；仅当元素与模数互质时才存在逆元。

- `DMod<ID>::set_mod(m)`：设置模数，要求 `m>0`；同一 `ID` 的全部对象共享模数。
- 不同动态模数同时存在时使用不同 `ID`，例如 `DMod<0>` 和 `DMod<1>`。
- `inv()` 会断言 gcd 为 `1`；除法前必须确认逆元存在。

乘法和构造需要一次 `i128 % ll`，比固定模数类慢，但不依赖浮点近似。

```
template<int ID = 0> struct DMod {
    inline static ll MOD = 1;
    ll x = 0;
    static void set_mod(ll m) { assert(m > 0), MOD = m; }
    static ll norm(i128 v) {
        v %= MOD;
        if (v < 0) v += MOD;
        return v;
    }
    DMod(ll v = 0) : x(norm(v)) {}
    ll val() const { return x; }
    static ll exgcd(ll a, ll b, i128 &u, i128 &v) {
        if (!b) return u = 1, v = 0, a;
        i128 x, y;
        ll g = exgcd(b, a % b, x, y);
        u = y, v = x - (i128)(a / b) * y;
        return g;
    }
    DMod pow(ll e) const { // e>=0
        DMod a = *this, r = 1;
        for (; e; e >>= 1, a *= a) if (e & 1) r *= a;
        return r;
```

```
    }
    DMod inv() const { // gcd(x,MOD)=1
        assert(MOD > 1 && x != 0);
        i128 u, v;
        ll g = exgcd(x, MOD, u, v);
        assert(g == 1);
        return DMod(norm(u));
    }
    DMod operator-() const { return DMod(x ? MOD - x : 0); }
    DMod &operator+=(const DMod &o) {
        x = x >= MOD - o.x ? x - (MOD - o.x) : x + o.x;
        return *this;
    }
    DMod &operator-=(const DMod &o) {
        x = x < o.x ? MOD - (o.x - x) : x - o.x;
        return *this;
    }
    DMod &operator*=(const DMod &o) { x = (i128)x * o.x % MOD; return *this; }
    DMod &operator/=(const DMod &o) { return *this *= o.inv(); }
    friend DMod operator+(DMod a, const DMod &b) { return a += b; }
    friend DMod operator-(DMod a, const DMod &b) { return a -= b; }
    friend DMod operator*(DMod a, const DMod &b) { return a *= b; }
    friend DMod operator/(DMod a, const DMod &b) { return a /= b; }
    friend bool operator==(const DMod &a, const DMod &b) { return a.x == b.x; }
    friend bool operator!=(const DMod &a, const DMod &b) { return a.x != b.x; }
    friend istream &operator>>(istream &in, DMod &a) { ll v; in >> v; a = DMod(v);
return in; }
    friend ostream &operator<<(ostream &out, const DMod &a) { return out << a.x; }
};
```

模数设为 `1` 时唯一规范值是 `0`，但任何元素都没有乘法逆元。设置新模数后，旧对象保存的值不会自动重新归一化，因此不能继续使用。

---

5. 精确分数

5.1 `ll` 分数 `Frac`

始终保持 `q>0`、`gcd(|p|,q)=1`。乘除前交叉约分，加减先约分分母 gcd，能显著降低中间值；最终乘加和比较使用 `i128`。

- `Frac(p,q)` 表示 `p/q`，要求 `q!=0`。
- 支持四则运算、比较、负号和输入输出；输入格式可为 `a` 或 `a/b`。
- `floor()`、`ceil()` 返回数学意义的向下/向上取整。
- 最终最简分子分母仍必须能放进 `ll`，否则断言失败并应改用 `BigFrac`。

```
struct Frac {
    ll p = 0, q = 1;
    static ll cast(i128 x) {
```

```
        assert(x >= numeric_limits<ll>::min() && x <= numeric_limits<ll>::max());
        return (ll)x;
    }
    Frac(ll p = 0, ll q = 1) : p(p), q(q) {
        assert(q != 0);
        norm();
    }
    void norm() {
        if (q < 0) p = -p, q = -q;
        ll g = gcd(p, q);
        p /= g, q /= g;
        if (!p) q = 1;
    }
    Frac operator-() const { return {-p, q}; }
    Frac &operator+=(const Frac &o) {
        ll g = gcd(q, o.q), a = q / g, b = o.q / g;
        p = cast((i128)p * b + (i128)o.p * a);
        q = cast((i128)a * o.q);
        norm();
        return *this;
    }
    Frac &operator-=(const Frac &o) { return *this += -o; }
    Frac &operator*=(const Frac &o) {
        ll a = p, b = o.p, c = q, d = o.q;
        ll g1 = gcd(a < 0 ? -a : a, d), g2 = gcd(b < 0 ? -b : b, c);
        a /= g1, d /= g1, b /= g2, c /= g2;
        p = cast((i128)a * b), q = cast((i128)c * d);
        norm();
        return *this;
    }
    Frac &operator/=(const Frac &o) {
        assert(o.p != 0);
        return *this *= Frac(o.q, o.p);
    }
    friend Frac operator+(Frac a, const Frac &b) { return a += b; }
    friend Frac operator-(Frac a, const Frac &b) { return a -= b; }
    friend Frac operator*(Frac a, const Frac &b) { return a *= b; }
    friend Frac operator/(Frac a, const Frac &b) { return a /= b; }
    friend bool operator==(const Frac &a, const Frac &b) { return a.p == b.p && a.q
== b.q; }
    friend bool operator!=(const Frac &a, const Frac &b) { return !(a == b); }
    friend bool operator<(const Frac &a, const Frac &b) { return (i128)a.p * b.q <
(i128)b.p * a.q; }
    friend bool operator>(const Frac &a, const Frac &b) { return b < a; }
    friend bool operator<=(const Frac &a, const Frac &b) { return !(b < a); }
    friend bool operator>=(const Frac &a, const Frac &b) { return !(a < b); }
    ll floor() const { // floor(p/q)
        ll x = p / q;
        return p < 0 && p % q ? x - 1 : x;
    }
    ll ceil() const { // ceil(p/q)
        ll x = p / q;
```

```
        return p > 0 && p % q ? x + 1 : x;
    }
    long double val() const { return (long double)p / q; }
    friend istream &operator>>(istream &in, Frac &x) {
        string s; in >> s;
        size_t k = s.find('/');
        x = k == string::npos ? Frac(stoll(s))
                              : Frac(stoll(s.substr(0, k)), stoll(s.substr(k + 1)));
        return in;
    }
    friend ostream &operator<<(ostream &out, const Frac &x) {
        out << x.p;
        if (x.q != 1) out << '/' << x.q;
        return out;
    }
};
```

为保持短小，代码假定构造参数不是 `LLONG_MIN`；该值取绝对值或翻转符号本身无法放进 `ll`。存在这种输入时直接使用 `BigFrac`。

5.2 高精度分数 `BigFrac`

分子分母使用 `cpp_int`，所有运算精确且无固定范围。依然在乘除前交叉约分，避免分子分母无意义膨胀。

```
struct BigFrac {
    cpp_int p = 0, q = 1;
    static cpp_int ab(cpp_int x) { return x < 0 ? -x : x; }
    static cpp_int gc(cpp_int a, cpp_int b) {
        a = ab(a), b = ab(b);
        while (b != 0) a %= b, swap(a, b);
        return a;
    }
    BigFrac(cpp_int p = 0, cpp_int q = 1) : p(move(p)), q(move(q)) {
        assert(this->q != 0);
        norm();
    }
    void norm() {
        if (q < 0) p = -p, q = -q;
        cpp_int g = gc(p, q);
        p /= g, q /= g;
        if (p == 0) q = 1;
    }
    BigFrac operator-() const { return {-p, q}; }
    BigFrac &operator+=(const BigFrac &o) {
        cpp_int g = gc(q, o.q), a = q / g, b = o.q / g;
        p = p * b + o.p * a, q = a * o.q;
        norm();
        return *this;
    }
    BigFrac &operator-=(const BigFrac &o) { return *this += -o; }
```

```
    BigFrac &operator*=(const BigFrac &o) {
        cpp_int a = p, b = o.p, c = q, d = o.q;
        cpp_int g1 = gc(a, d), g2 = gc(b, c);
        a /= g1, d /= g1, b /= g2, c /= g2;
        p = a * b, q = c * d;
        norm();
        return *this;
    }
    BigFrac &operator/=(const BigFrac &o) {
        assert(o.p != 0);
        return *this *= BigFrac(o.q, o.p);
    }
    friend BigFrac operator+(BigFrac a, const BigFrac &b) { return a += b; }
    friend BigFrac operator-(BigFrac a, const BigFrac &b) { return a -= b; }
    friend BigFrac operator*(BigFrac a, const BigFrac &b) { return a *= b; }
    friend BigFrac operator/(BigFrac a, const BigFrac &b) { return a /= b; }
    friend bool operator==(const BigFrac &a, const BigFrac &b) { return a.p == b.p
&& a.q == b.q; }
    friend bool operator!=(const BigFrac &a, const BigFrac &b) { return !(a == b); }
    friend bool operator<(const BigFrac &a, const BigFrac &b) { return a.p * b.q <
b.p * a.q; }
    friend bool operator>(const BigFrac &a, const BigFrac &b) { return b < a; }
    friend bool operator<=(const BigFrac &a, const BigFrac &b) { return !(b < a); }
    friend bool operator>=(const BigFrac &a, const BigFrac &b) { return !(a < b); }
    cpp_int floor() const {
        cpp_int x = p / q;
        return p < 0 && p % q != 0 ? x - 1 : x;
    }
    cpp_int ceil() const {
        cpp_int x = p / q;
        return p > 0 && p % q != 0 ? x + 1 : x;
    }
    friend istream &operator>>(istream &in, BigFrac &x) {
        string s; in >> s;
        size_t k = s.find('/');
        x = k == string::npos ? BigFrac(cpp_int(s))
                              : BigFrac(cpp_int(s.substr(0, k)), cpp_int(s.substr(k
+ 1)));
        return in;
    }
    friend ostream &operator<<(ostream &out, const BigFrac &x) {
        out << x.p;
        if (x.q != 1) out << '/' << x.q;
        return out;
    }
};
```

`BigFrac` 适合精确几何斜率、概率、连分数和符号计算。若只需要最终比较，尽量约分或使用专门的交叉乘比较，避免每一步都构造巨大分数。

6. 高精度浮点与精确十进制

6.1 `cpp_dec_float`

`cpp_dec_float<N>` 是约 `N` 位十进制有效数字的浮点数，支持 `sqrt/exp/log/sin/cos/atan2` 等数学函数。它仍有舍入误差，不能代替精确分数。

```
using F100 =
boost::multiprecision::number<boost::multiprecision::cpp_dec_float<100>>;

F100 a, b;
cin >> a >> b;
F100 c = sqrt(a) + exp(b);
F100 pi = acos(F100(-1));
cout << fixed << setprecision(80) << c << '\n';

cpp_int x = cpp_int(1) << 500;
F100 fx(x.convert_to<string>()); // 大整数通过十进制字符串转高精浮点
```

模板位数是编译期常量。位数越高，四则运算和初等函数越慢；题目只要求 `1e-12` 时优先使用 `long double`。

6.2 十进制字符串精确转 `BigFrac`

浮点读入无法精确保留 `0.1`。若题目要求十进制输入完全精确，可解析为整数乘 `10` 的幂。本函数支持符号、小数点和科学计数法，如 `-12.340e-2`。

```
BigFrac decimal_frac(string s) { // 精确解析十进制/科学计数法
    int sg = 1, p = 0, e = 0;
    if (s[p] == '+' || s[p] == '-') sg = s[p++] == '-' ? -1 : 1;
    size_t ep = s.find_first_of("eE", p);
    string m = ep == string::npos ? s.substr(p) : s.substr(p, ep - p);
    if (ep != string::npos) e = stoi(s.substr(ep + 1));
    size_t dot = m.find('.');
    int frac = 0;
    if (dot != string::npos) frac = (int)m.size() - dot - 1, m.erase(dot, 1);
    cpp_int x = 0;
    for (char c : m) x = x * 10 + c - '0';
    if (sg < 0) x = -x;
    int k = e - frac;
    if (k >= 0) return BigFrac(x * pow_big(cpp_int(10), k));
    return BigFrac(x, pow_big(cpp_int(10), -k));
}
```

该函数依赖 2.2 节的 `pow_big(cpp_int,uint64_t)` 与 5.2 节的 `BigFrac`。指数 `e` 使用 `int`，适合竞赛常见输入。指数达到数十万时，结果本身就需要相应数量的数位，内存和时间无法绕过。

---

7. 常见结论与性能取舍

7.1 范围与溢出

- `ll` 最大约 `9.22e18`，两个 `1e18` 相乘必须先转 `i128`。
- `i128` 最大约 `1.70e38`，能容纳任意两个 `ll` 的乘积，但不一定容纳三个。
- 表达式在赋值前已经计算；写 `i128 x=a*b` 时若 `a,b` 是 `ll`，乘法仍可能先溢出。必须写 `(i128)a*b`。
- 有符号整数溢出是未定义行为，不能依靠自然回绕。

7.2 除法、余数和取模

C++ 有符号整数 `/` 向 `0` 截断，`%` 与被除数同号；数学题中的 `floor(a/b)` 和非负模需要显式修正。

```
ll norm_mod(ll x, ll mod) { // mod>0，返回 [0,mod)
    x %= mod;
    return x < 0 ? x + mod : x;
}
```

`a/b mod MOD` 表示 `a*inv(b)`，只在 `b` 有模逆时成立。质数模下也必须保证 `b%MOD!=0`；合数模下要求 `gcd(b,MOD)=1`。

7.3 大整数复杂度

设高精度数含 `n` 个机器字或基数块：

操作	典型复杂度
比较、加减	`O(n)`
乘小整数、除小整数	`O(n)`
自写 `Big` 大整数乘法	`O(nm)`
自写 `Big` 长除法	`O(nm)`
`cpp_int` 大数乘法	按规模自动选择朴素/Karatsuba 等算法
gcd	多次除法，输入接近 Fibonacci 数时轮数较多

十进制位数本身就是输出规模；要求输出百万位答案至少需要 `O(百万)` 时间和空间。

7.4 分数大小控制

- 每次运算后约分，乘除前交叉约分。
- 连续相加时可先求分母 gcd，避免直接形成分母乘积。
- 只做比较时使用交叉乘，不必构造新分数；`ll` 分数比较应转 `i128`。
- 分母统一为正，才能让比较、floor 和输出规则一致。
- 高精度分数的分子分母可能指数增长；“不会溢出”不等于“不会超时”。

7.5 浮点比较

高精度浮点也不能直接代表所有实数。一般使用相对误差与绝对误差共同判断：

```
bool eq(long double a, long double b) {
    const long double eps = 1e-15L;
    return fabsl(a - b) <= eps * max({1.0L, fabsl(a), fabsl(b)});
}
```

几何方向、分数大小、整数平方判定能用精确整数时，优先使用 `i128/cpp_int`，不要靠扩大浮点精度掩盖建模问题。

---

8. 赛场检查清单

1. 普通 `ll` 是否足够，乘法前是否已经转换为 `i128`。
2. 题目需要向 `0` 截断、向下取整还是向上取整。
3. 负数取模是否已经归一化到 `[0,mod)`。
4. 模数是否为质数，除数是否确实存在模逆。
5. 修改 `DMod<ID>` 模数后是否还错误使用了旧对象。
6. `cpp_int` 转回内置整数前是否检查范围。
7. `msb(x)` 是否保证 `x>0`，除法和取模是否保证除数非零。
8. 分数分母是否非零、是否保持为正、最终分子分母是否仍能放进 `ll`。
9. 自写 `Big` 的商余语义是否符合题目对负数的要求。
10. 高精度浮点是近似值；题目是否实际要求精确十进制或精确分数。
11. 输出位数是否本身已经超过合理的时间与内存限制。
12. 当前编译环境是否提供 Boost；若没有，是否已经改用自写 `Big` 并删除 Boost 头文件。